

**A
Project Report
On
"Cloud Based
Document Management
Framework"**



(CE447 - Software Project Major)

Prepared by
Tirthkumar Shah (17DCE065)

Under the Supervision of
Prof. Khushi Patel

Submitted to
Charotar University of Science & Technology (CHARUSAT)
for the Partial Fulfillment of the Requirements for the
Degree of Bachelor of Technology (B.Tech.)
in Computer Engineering (CE)
for 8th semester B. Tech
Submitted at



DEPARTMENT OF COMPUTER ENGINEERING
Devang Patel Institute of Advance Technology and Research (DEPSTAR)
Faculty of Technology & Engineering (FTE), CHARUSAT
At: Changa, Dist: Anand, Pin: 388421.
April, 2021



Accredited with Grade A by NAAC
Accredited with Grade A by KCG

CERTIFICATE

This is to certify that the report entitled “**Cloud Based Document Management Framework**” is a bonafied work carried out by **Tirthkumar Shah (17DCE065)** under the guidance and supervision of **Prof. Khushi Patel & Mr Krishnakumar V. G.** for the subject **Software Project Major (CE447)** of 8th Semester of Bachelor of Technology in **Computer Engineering** at Devang Patel Institute of Advance Technology and Research (DEPSTAR), Faculty of Technology & Engineering (FTE) – CHARUSAT, Gujarat.

To the best of my knowledge and belief, this work embodies the work of candidate himself, has duly been completed, and fulfills the requirement of the ordinance relating to the B.Tech. Degree of the University and is up to the standard in respect of content, presentation and language for being referred by the examiner(s).

Under the supervision of,

Ms. Khushi Patel,
Assistant Professor cum Research Fellow,
Department of Computer Engineering,
DEPSTAR-FTE, CHARUSAT, Changa,
Gujarat

Mr. Krishnakumar V.G,
Senior Consultant,
BFSI UK2 Tag,
Tata Consultancy Services,
Bangalore

Dr. Dweepna Garg,
Head- Department of Computer Engineering,
DEPSTAR-FTE,
CHARUSAT, Changa, Gujarat

Dr. (Prof.) Amit Ganatra
Principal-DEPSTAR,
Dean - FTE,
CHARUSAT, Changa, Gujarat.

**Devang Patel Institute of Advance Technology and Research
(DEPSTAR)**

Faculty of Technology & Engineering (FTE), CHARUSAT

At: Changa, Ta. Petlad, Dist. Anand, Pin:388421. Gujarat.

Acknowledgement

- The Success and final outcome of this project required a lot of guidance and assistance from many people and I am extremely privileged to have got this all along the completion of my project. All that I have done is only due to such supervision and assistance and we would not forget to thank them.
- I am extremely thankful to Prof. Khushi Patel and Mr. Krishnakumar V. G. for providing such a nice support and guidance, although they had busy schedule managing the college and company work. I owe my deep gratitude to my team members, who took keen interest on project work and guided me all along, till the completion of my project work by providing all the necessary information for developing a good project. Also encouraged me till the completion of my project work.

Abstract

- Today Developers around the world are making efforts to enhance user experience of using application as well as they are trying to enhance the developer's workflow of designing applications to deliver projects and rollout change requests under strict timeline. Stacks can be used to build web applications in the shortest span of time. The advantage of such JavaScript stacks helps to build an integrated solution by using only JavaScript. This project provides the introduction to concept and describes the MEAN stack open source.
- The stacks used in web development are basically the response of software engineers to current demands. They have essentially adopted pre-existing frameworks (including JavaScript) to make their lives easier. While there are many, MEAN and MERN are just two of the popular stacks that have evolved out of JavaScript. Both of these stacks are made up of open-source components and offer an end-to-end framework for building comprehensive web apps that enable browsers to connect with databases. The common theme between the two is JavaScript and this is also the key benefit of using either stack. You can basically avoid any syntax errors or any confusion by just coding in one programming language, JavaScript. Another advantage of building your next web project with MEAN or MERN is the fact that you benefit from its enhanced flexibility.
- The term MEAN stack refers to a collection of JavaScript based technologies used to develop web applications. MEAN is an acronym for MongoDB, ExpressJS, Angular and NodeJS from client to server to database, MEAN is full stack JavaScript.
- MongoDB is a schema less NoSQL database system. Express is lightweight framework used to build web applications in Node. Angular is an open-source, front end, JavaScript library for building user interfaces or UI components. Node.js is a server-side JavaScript execution environment.
- Overall, using the MEAN stack enables developers to build highly efficient web applications.

Table of Contents

Acknowledgement	1
Abstract.....	2
Table of Figures.....	4
Chapter 1 Introduction.....	5
1.1 Problem Statement.....	5
1.2 Project Definition.....	6
1.3 Objective	7
Chapter 2 Project Planning.....	8
2.1 Gantt Chart	9
Chapter 3 Technologies and Tools Used:.....	10
3.1 Technologies	10
3.2 Tools	12
Chapter 4 Requirements	13
4.1 Functional Requirements	13
4.2 Non-Functional Requirements.....	16
4.3 Hardware Requirements	17
Chapter 5 Implementation	18
5.1 Frontend.....	18
5.2 Backend.....	26
5.2.1 Key Topics	27
5.3 Code Management- Azure DevOps	33
5.3.1 Setup of Azure DevOps	34
5.4 Docker Containerization	36
Chapter 6 UML Diagrams	39
6.1 System Design.....	39
6.2 Use Case Diagram	40
6.3 Class Diagram	41
6.4 Activity Diagram.....	42
Chapter 7 Conclusion	43
Chapter 8 References.....	44

Table of Figures

Figure 1 Gantt Chart.....	9
Figure 2 Frontend Technologies	10
Figure 3 Node,Mongo,Express	11
Figure 4 Tools.....	12
Figure 5 Upload Functionality	13
Figure 6 Delete Functionality	14
Figure 7 Preview File	15
Figure 8 Folder Creation	15
Figure 9 Registration	18
Figure 10 Main Window-Read Access.....	18
Figure 11 Search Filter	19
Figure 12 Refresh Feature.....	19
Figure 13 Main Window-Write Access.....	20
Figure 14 Creation of Folder	20
Figure 15 Upload Files Dialog.....	21
Figure 16 Drag and Drop Feature.....	21
Figure 17 Window of Uploading Files	22
Figure 18 Navigate through Folders.....	22
Figure 19 Administrator Interface	23
Figure 20 Delete Feature	23
Figure 21 Download Files	24
Figure 22 Logout Window.....	25
Figure 23 Implementation Diagram.....	26
Figure 24 Node Module-Multer S3	27
Figure 25 Backend-Project Structure	27
Figure 26 Application Programming Layer.....	28
Figure 27 Robo3T Database Snapshot	29
Figure 28 Metadata of Amazon S3.....	30
Figure 29 Postman-API Testing	31
Figure 30 Deleted File Action	32
Figure 31 Azure DevOps.....	33
Figure 32 Azure Setup.....	34
Figure 33 Project Home Window.....	34
Figure 34 Azure Repository	35
Figure 35 Commit History	35
Figure 36 Docker Introduction.....	36
Figure 37 Docker Toolbox	37
Figure 38 Containers	37
Figure 39 Docker Compose Up	38
Figure 40 Multi-Containers	38
Figure 41 System Design	39
Figure 42 Use Case	40
Figure 43 Class Diagram	41
Figure 44 Activity Diagram	42

Chapter 1 Introduction

1.1 Problem Statement

- Company needs a document repository system for a particular client project. The project generates a large number of files that need to be labelled with specific metadata, and as there is no system in place to store the files or the corresponding metadata in a shared space, the team members are resorting to storing the files locally and forgoing the metadata.
- Company would like a document repository system, much in the vein of AWS S3, to be made for this project to fulfil the aforementioned purposes.

1.2 Project Definition

- The system should be able to upload files to a specific S3 bucket and ask for user input for certain metadata fields such as “Document Type” and “Remarks”.
- Users should also be able to view, download and delete files stored in the bucket.
- Optional features include a directory view to create one or more layers of folders in order to store files in an organized manner. One should also be able to navigate through this directory with ease.

1.3 Objective

- It is a Cloud Based Document Management Framework and Full Stack Web Application
- where Employee user can upload, delete and view the documents.
- DocR-Document Repository supports user registrations and login/logout features, and allows users to view, download and/or upload and delete files/folders depending on whether they have write access or not respectively.

Chapter 2 Project Planning

- For the project planning and management, we followed Agile Methodology.
- An agile software development process always starts by defining the users and documenting a vision statement on a scope of problems, opportunities, and values to be addressed. The product owner captures this vision and works with a multidisciplinary team (or teams) to deliver on this vision.
- Many agile frameworks that provide specifics on development processes and agile development practices, aligned to a software development life cycle.
- The most popular agile framework is called *scrum*. It focuses on a delivery cadence called a *sprint* and meeting structures that include the following:
 - Planning — where sprint priorities are identified
 - Commitment — where the team reviews a list or backlog of user stories and decides how much work can be done in the sprint's duration
 - Daily stand-up meetings — so teams can communicate updates on their development status and strategies)
- Sprints end with a demo meeting where the functionality is shown to the product owner, followed by a retrospective meeting where the team discusses what went well and what needs improvement in their process.

2.1 Gantt Chart

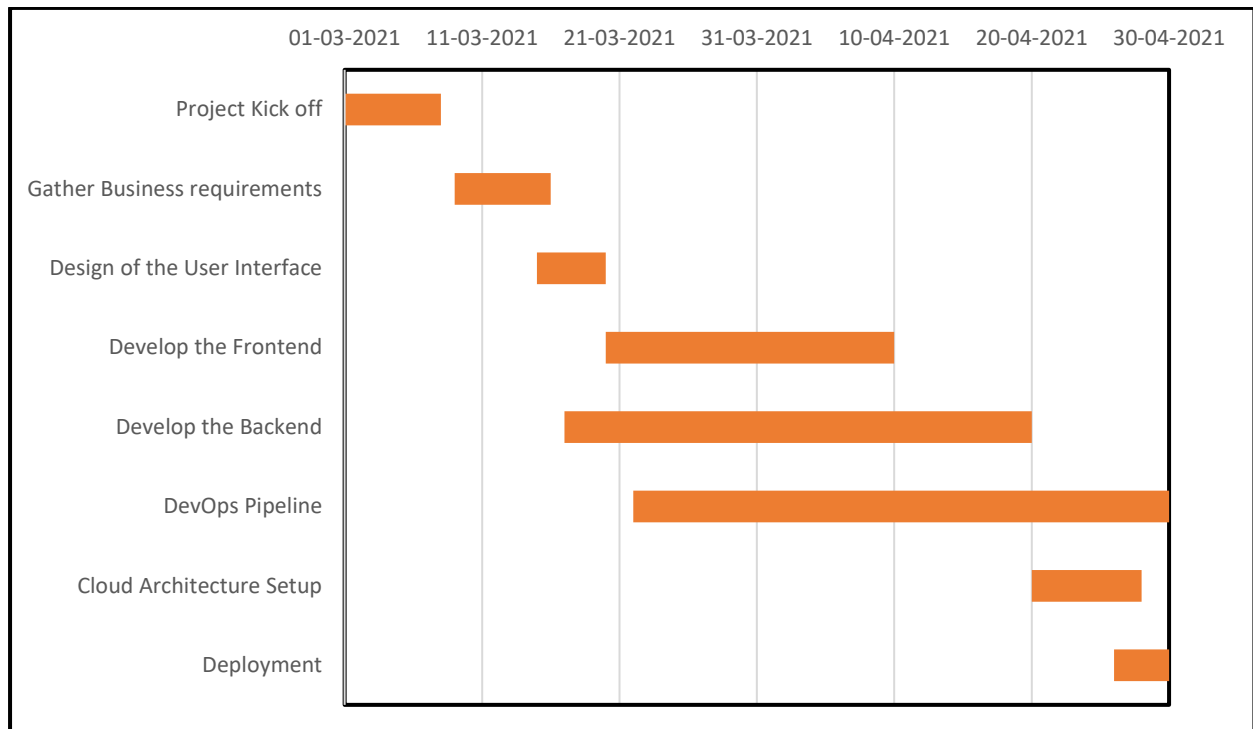


Figure 1 Gantt Chart

Chapter 3 Technologies and Tools Used:

3.1 Technologies

➤ Frontend

- **Angular 8:**

Angular is a platform and framework for building single-page client applications using HTML and TypeScript. Angular is written in TypeScript. It implements core and optional functionality as a set of TypeScript libraries that you import into your apps.

- **JavaScript (ES6)**

JavaScript, often abbreviated as JS, is a programming language that conforms to the ECMAScript specification. JavaScript is high-level, often just-in-time compiled, and multi-paradigm. It has curly-bracket syntax, dynamic typing, prototype-based object-orientation, and first-class functions.

- **HTML5, CSS3**

HTML (the Hypertext Markup Language) and CSS (Cascading Style Sheets) are two of the core technologies for building Web pages. HTML provides the structure of the page, CSS the (visual and aural) layout, for a variety of devices.



Figure 2 Frontend Technologies

➤ **Backend**

- **NodeJs**

Node.js is an open-source, cross-platform, back-end JavaScript runtime environment that runs on the V8 engine and executes JavaScript code outside a web browser.

- **Mongo DB**

MongoDB is a source-available cross-platform document-oriented database program. Classified as a NoSQL database program, MongoDB uses JSON-like documents with optional schemas.

- **AWS (Amazon Web Services) Services**

Amazon Web Services is a subsidiary of Amazon providing on-demand cloud computing platforms and APIs to individuals, companies, and governments, on a metered pay-as-you-go basis.

- **Azure DevOps**

Azure DevOps Server is a Microsoft product that provides version control, reporting, requirements management, project management, automated builds, testing and release management capabilities. It covers the entire application lifecycle, and enables DevOps capabilities.

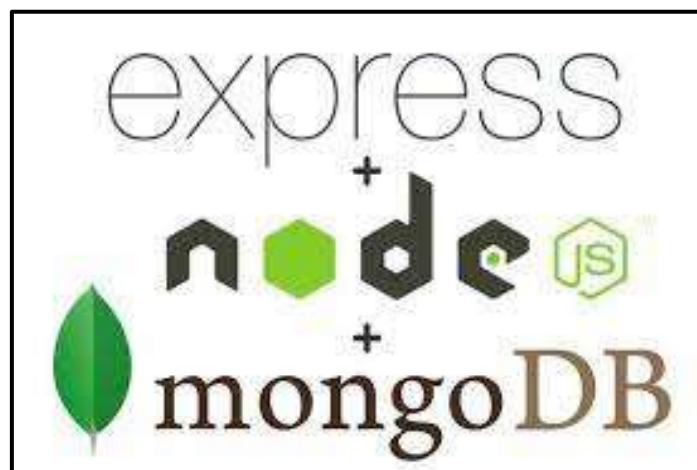


Figure 3 Node,Mongo,Express

3.2 Tools

➤ Code Editor

- **Visual Studio code**

Visual Studio Code is a freeware source-code editor made by Microsoft for Windows, Linux and macOS. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git.

➤ Design

- **Adobe XD**

Adobe XD is a vector-based user experience design tool for web apps and mobile apps, developed and published by Adobe Inc. It is available for macOS and Windows.

➤ API (Application Programming Interface) Testing

- **Postman**

Postman is a collaboration platform for API development. Postman's features simplify each step of building an API and streamline collaboration so you can create better APIs—faster.

➤ Code Management

- **Github**

It is a provider of Internet hosting for software development and version control using Git. It offers the distributed version control and source code management functionality of Git, plus its own features.



Figure 4 Tools

Chapter 4 Requirements

4.1 Functional Requirements

1. Upload File

- The main functionality of system is to upload the files in the Blob storage. Here, as a blob storage we have used Amazon S3 (Simple storage Service).
- If the user has write access or user is administrator, then and only then user can upload files.

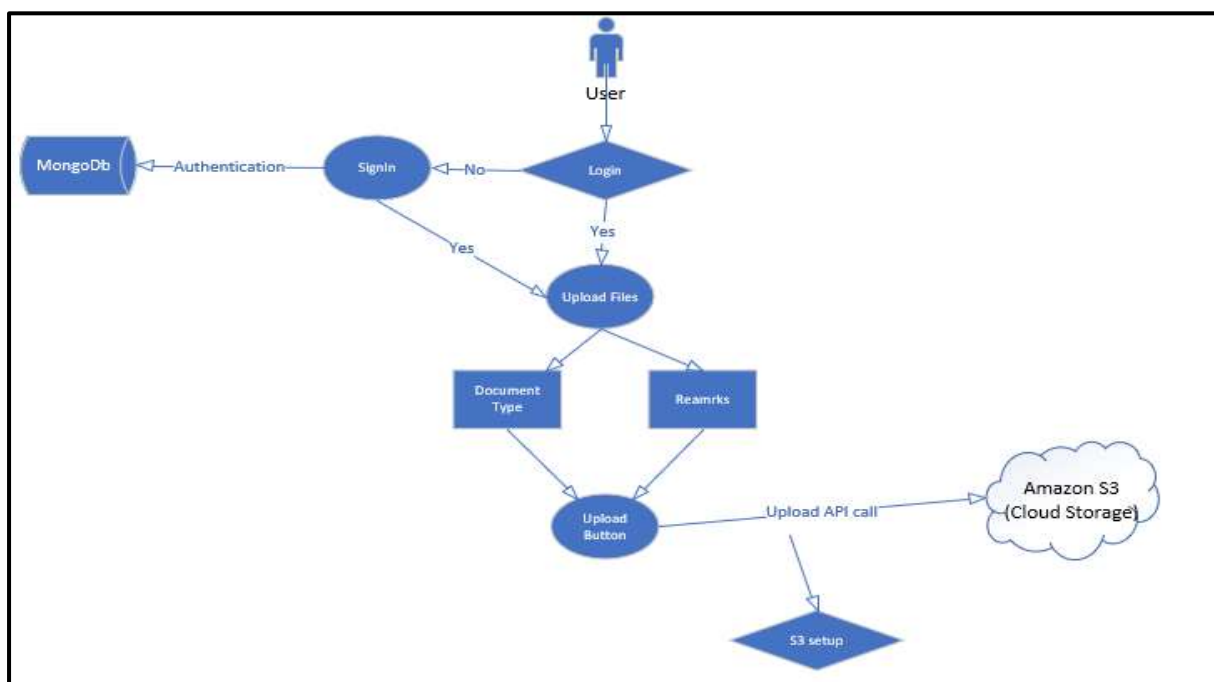


Figure 5 Upload Functionality

2. Delete File

- This functionality focuses on deletion of file from the storage.
- For delete action, user have to access the ownership of file, otherwise it cannot delete the file from the system except it has administrator power.
- Here, instead of deleting the file permanently, file will be moved to another blob storage and from there it will be delete after 7 days.

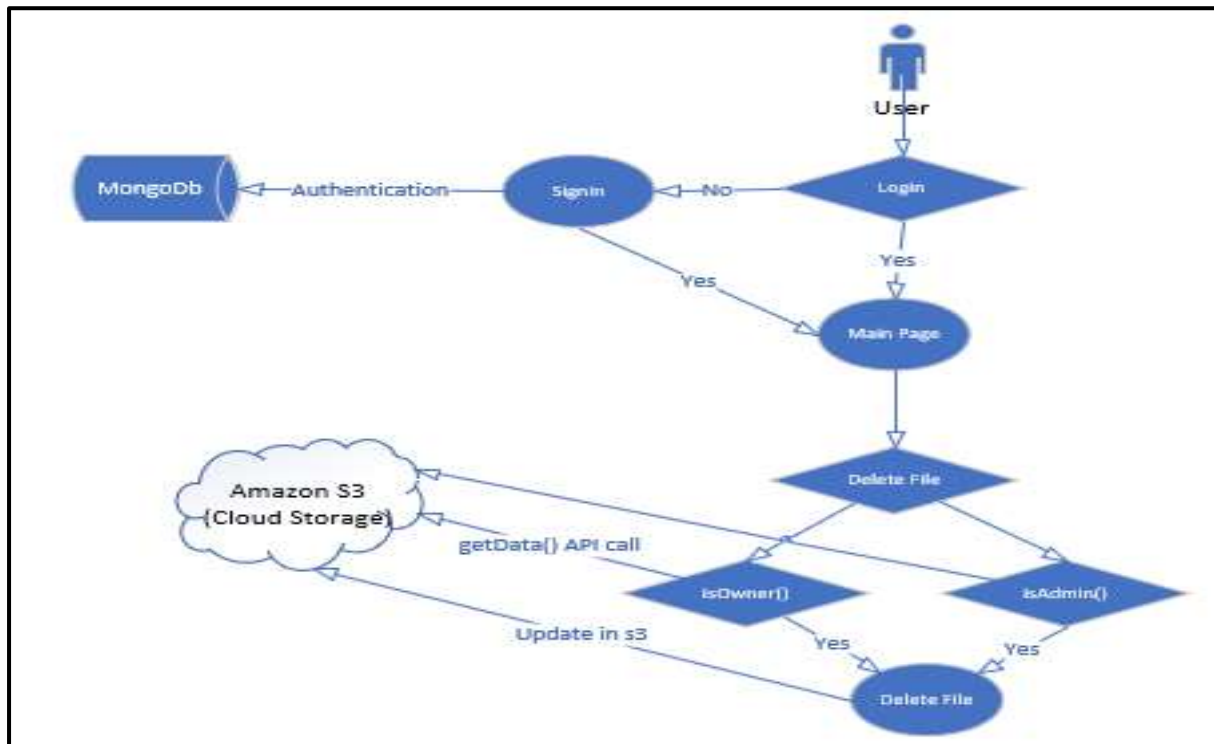


Figure 6 Delete Functionality

3. Download/View File

- The system has designed in such a way that every user has read access, so user will be able to download and view the files.

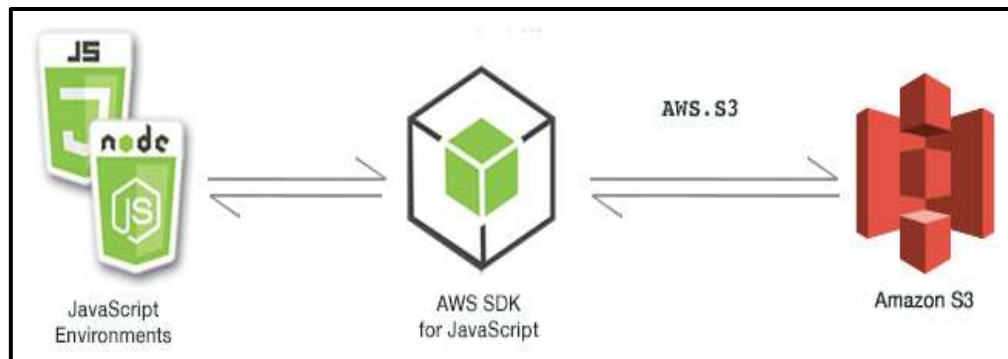


Figure 7 Preview File

4. Create Folder

- With the help of this feature, user will be able to create folder in the system. Also, we have programmed nested folder. So, user can make unlimited folder inside the folder.
- Here, User can create folder only if it has write access of the system.

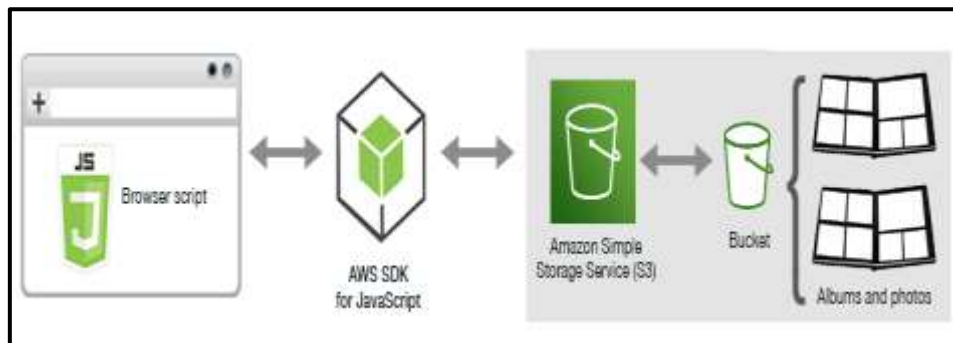


Figure 8 Folder Creation

5. Delete Folder

- In this functionality, only folder owner can delete the folder himself/herself.
- Another user will not be able to delete the folder, unless it has administrator access.

6. User Registration/Login

- Here, users can register themselves and got access to the system.

4.2 Non-Functional Requirements

1. Highly Scalable

- In today's world system should be run all the time, consider that point in mind, we created the cloud architecture in such a manner that I will be fault tolerance.
- Even if the system crash at some point, it will be automatically online within few minutes.
- Here, we implemented Highly Scalable cloud architecture.

2. Highly Available

- First of all, highly available means system's available time.
- We can calculate the Availability of system from below formula:
Availability= (Available Time) / (Total Time (24 hours)) X 100
- Our goal is to maintain system's availability as high as possible.

4.3 Hardware Requirements

- Here, mentioned below specifications are needed for smooth execution of the project.
- Windows 10
- Intel i7-7500u
- 8 GB RAM
- 256 GB SSD

Chapter 5 Implementation

5.1 Frontend

1. Register/Login Page:

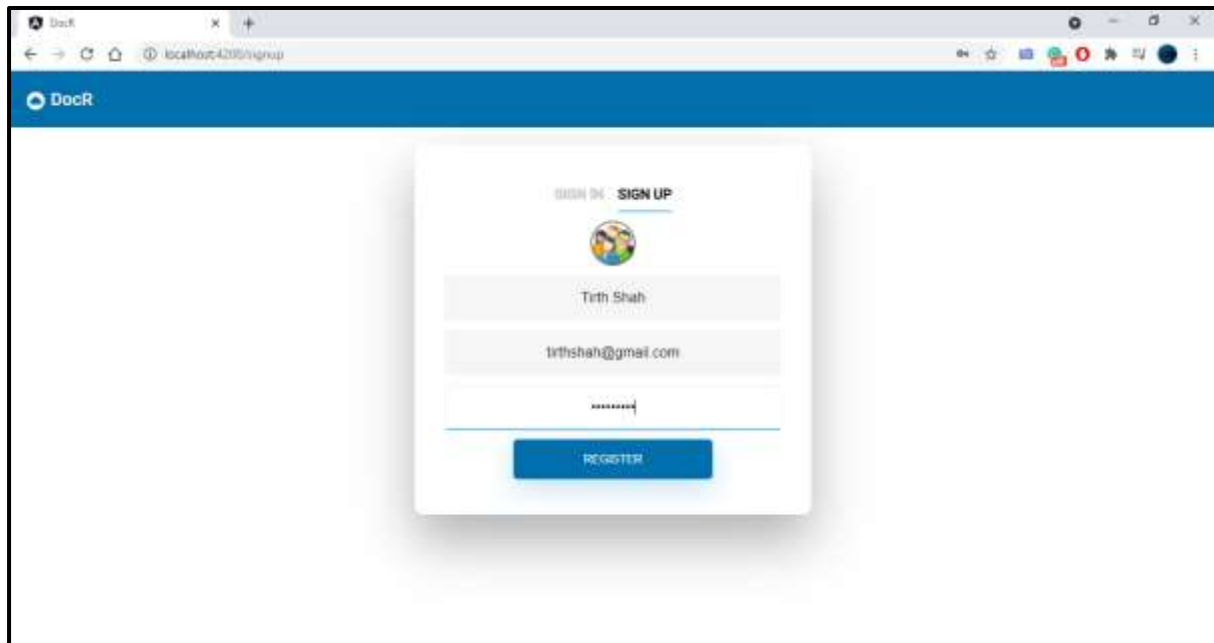


Figure 9 Registration

2. Main View (User Access: Read Only)

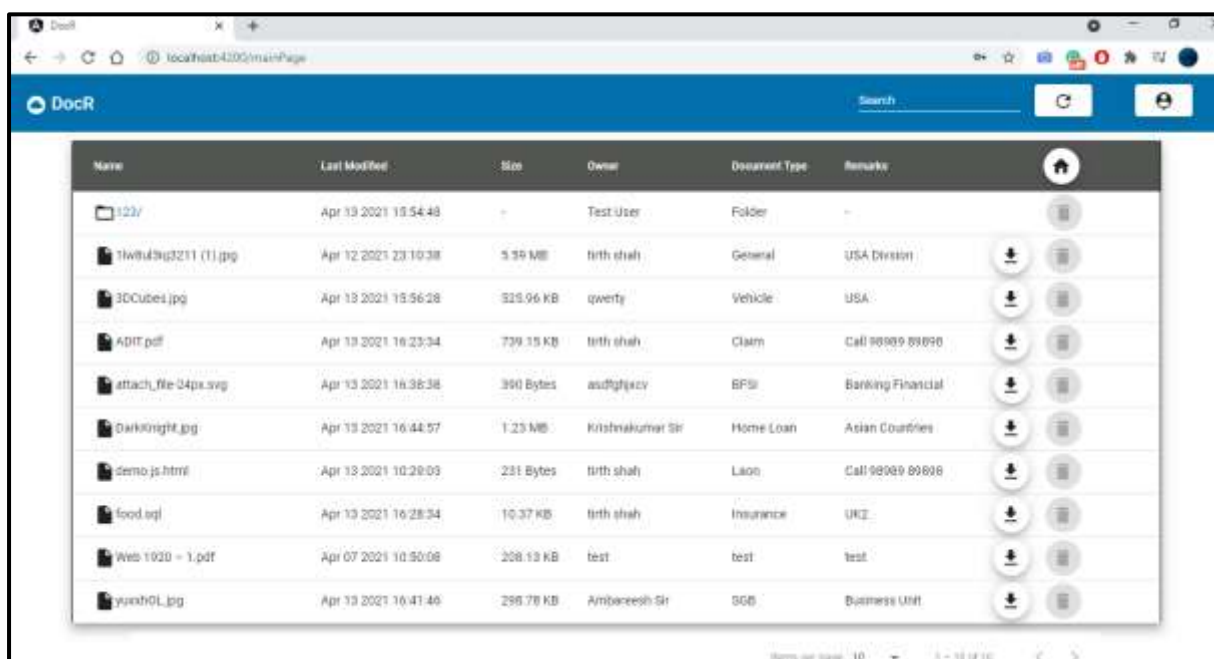


Figure 10 Main Window-Read Access

3. Search Function

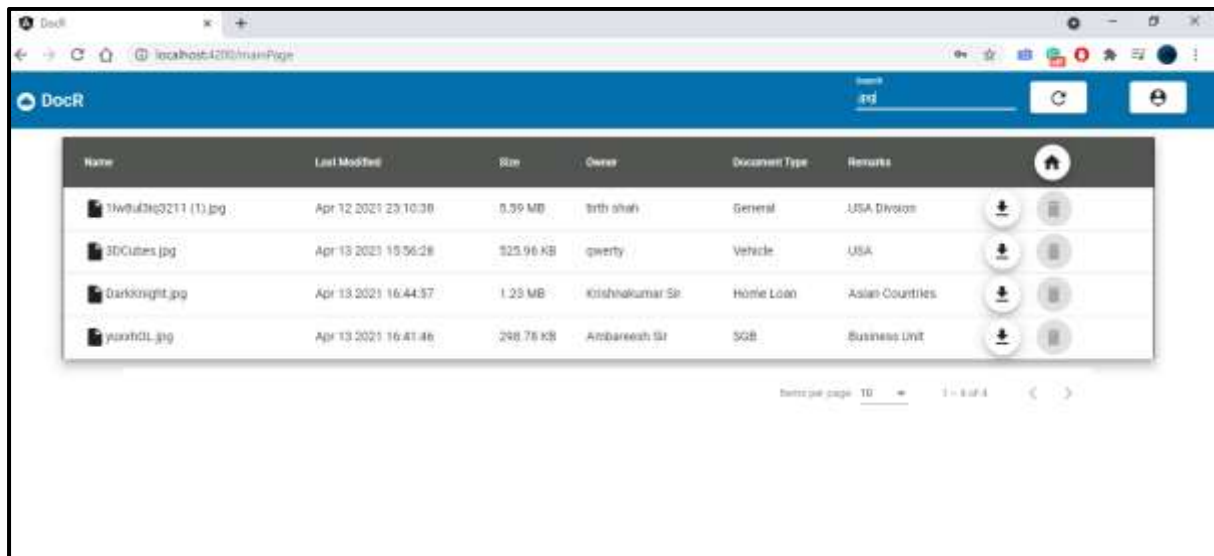


Figure 11 Search Filter

4. Refresh Button

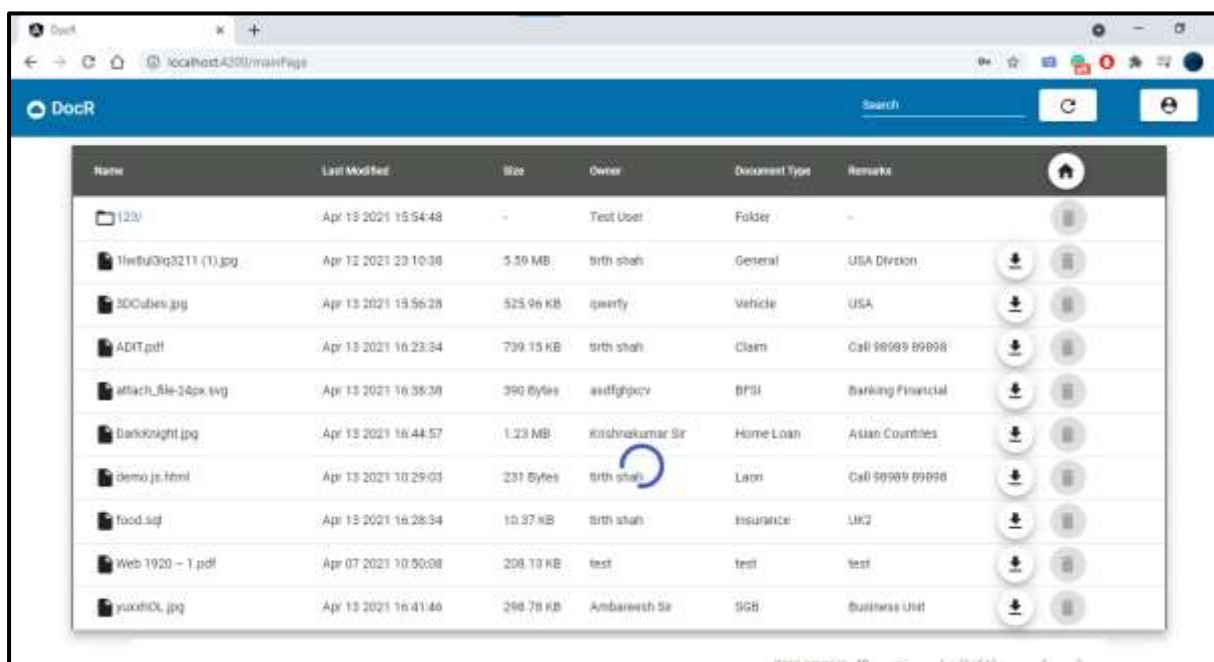


Figure 12 Refresh Feature

5. Main View (User Access: Read and Write):

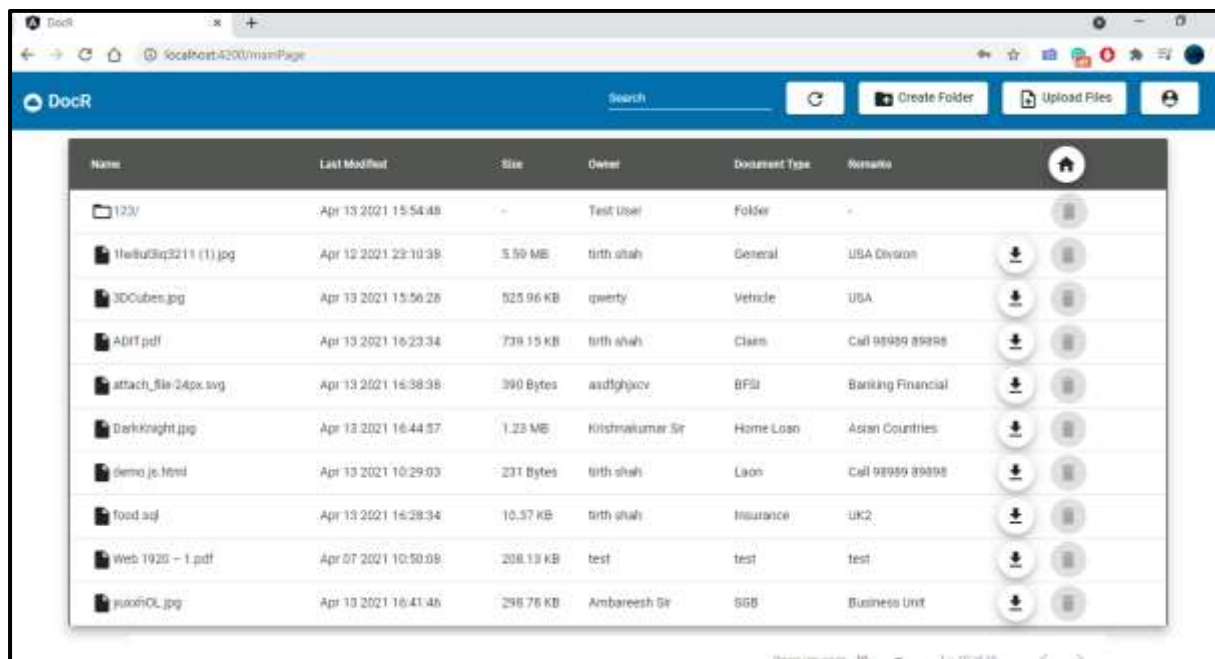


Figure 13 Main Window-Write Access

6. Create Folder Functionality:

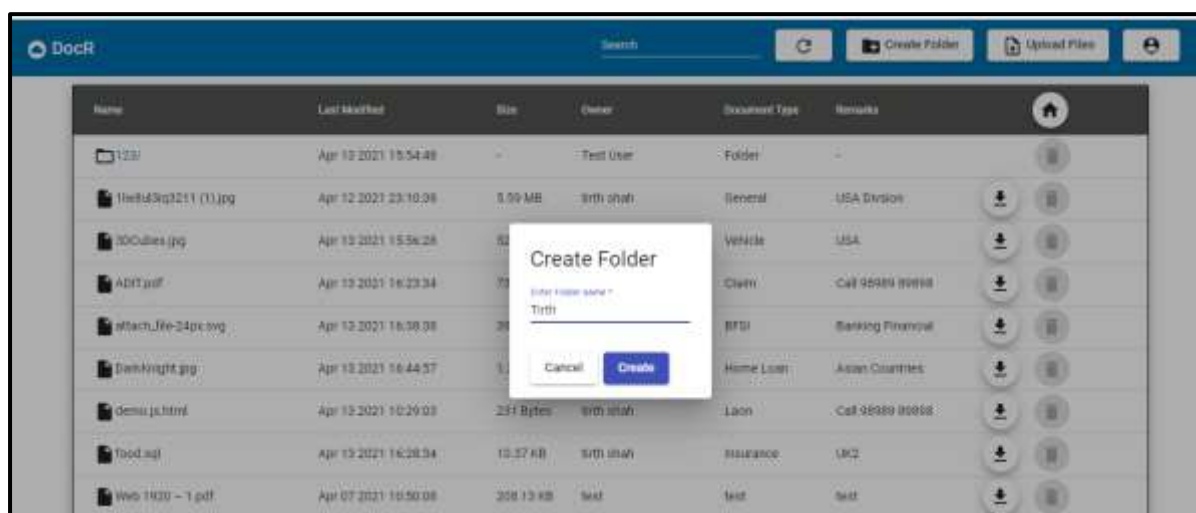


Figure 14 Creation of Folder

7. Upload File/Files

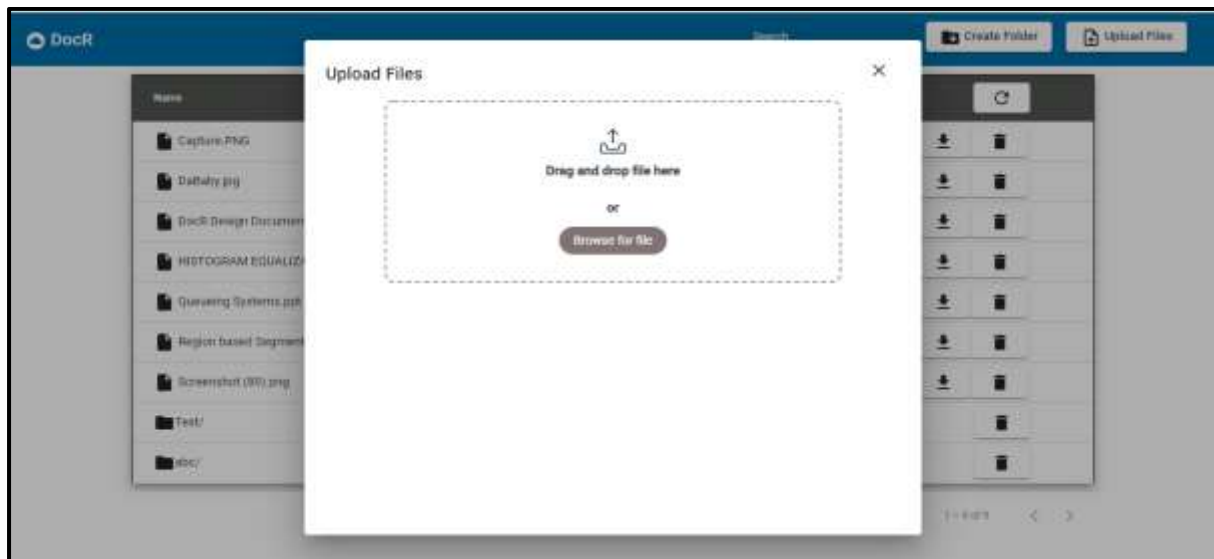


Figure 15 Upload Files Dialog

8. Drag and Drop Feature:

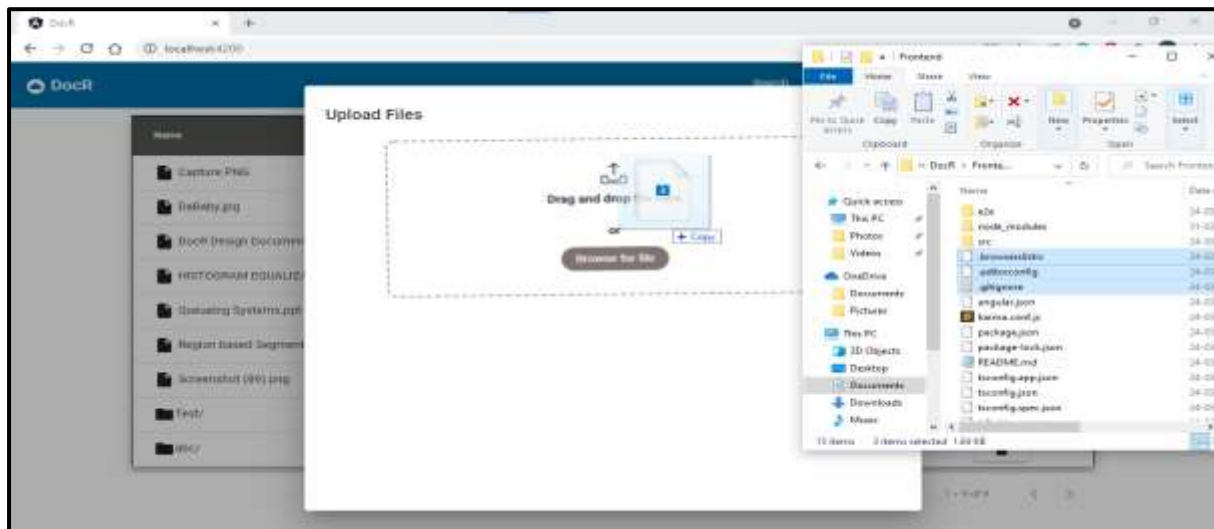


Figure 16 Drag and Drop Feature

9. While Uploading File:



Figure 17 Window of Uploading Files

10. Folder Navigation:

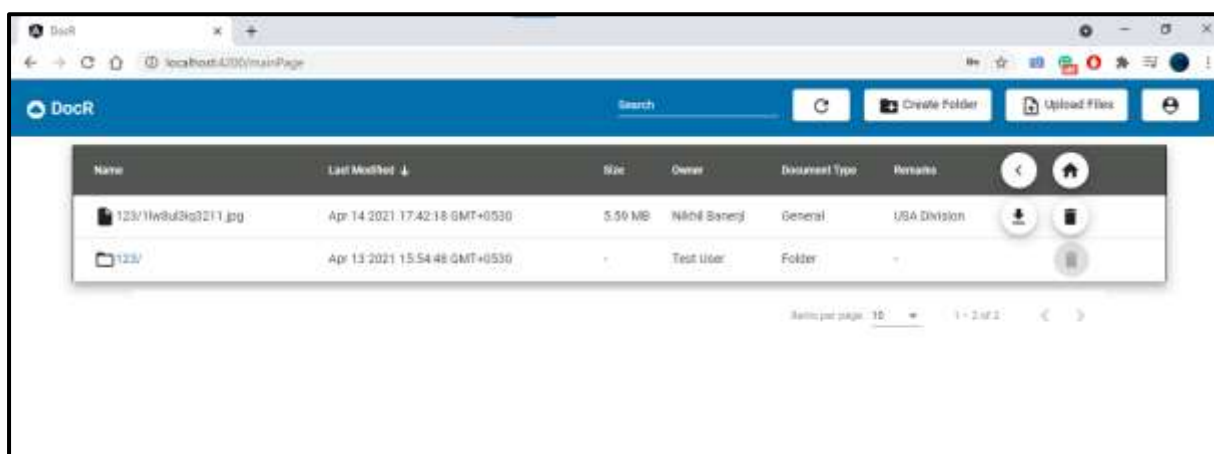


Figure 18 Navigate through Folders

11. Main View (User Access: Administrator)

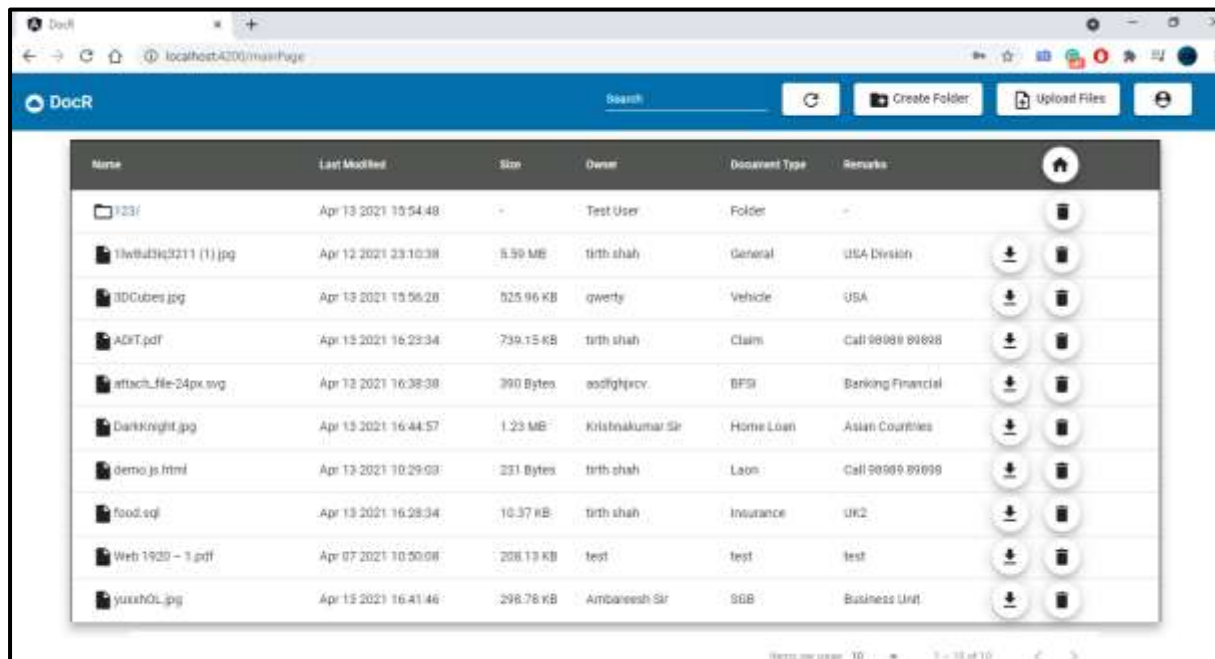


Figure 19 Administrator Interface

12. Delete Confirmation:

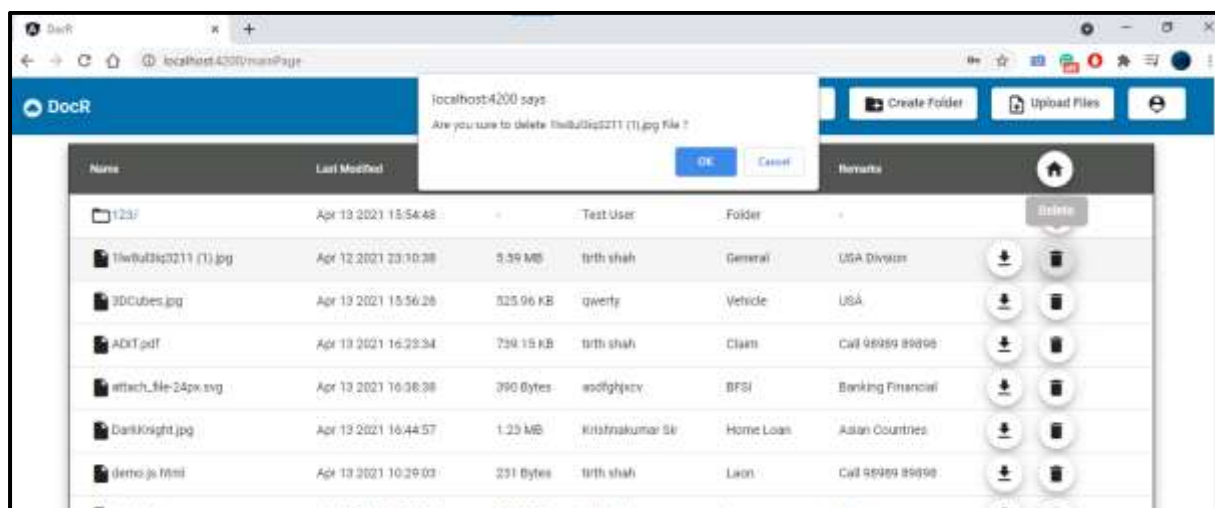


Figure 20 Delete Feature

13. Download/View Feature:

DocR		
Name	Last Modified	Size
 123/	Apr 13 2021 15:54:48	-
 1lw8ul3iq3211 (1).jpg	Apr 12 2021 23:10:38	5.59 MB
 3DCubes.jpg	Apr 13 2021 15:56:28	525.96 KB
 ADIT.pdf	Apr 13 2021 16:23:34	739.15 KB
 attach_file-24px.svg	Apr 13 2021 16:38:38	390 Bytes
 DarkKnight.jpg	Apr 13 2021 16:44:57	1.23 MB
 demo.js.html	Apr 13 2021 10:29:03	231 Bytes
 food.sql	Apr 13 2021 16:28:34	10.37 KB
 Web 1920 - 1.pdf	Apr 07 2021 10:50:08	208.13 KB
 yuxxhOL.jpg	Apr 13 2021 16:41:46	298.78 KB
 3DCubes (1).jpg ^		

Figure 21 Download Files

14. Logout Window:

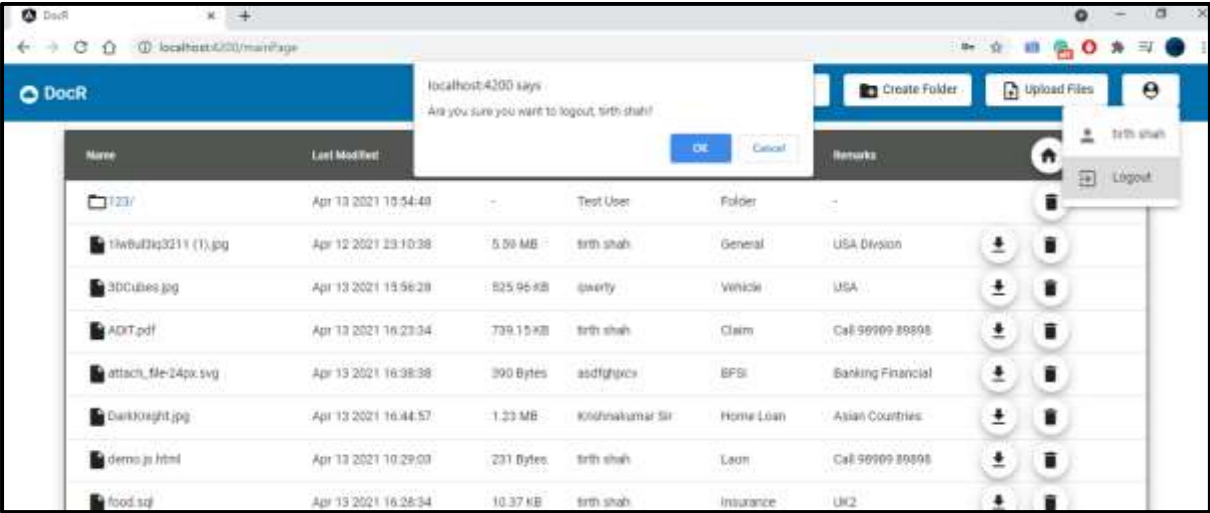


Figure 22 Logout Window

5.2 Backend

Implementation

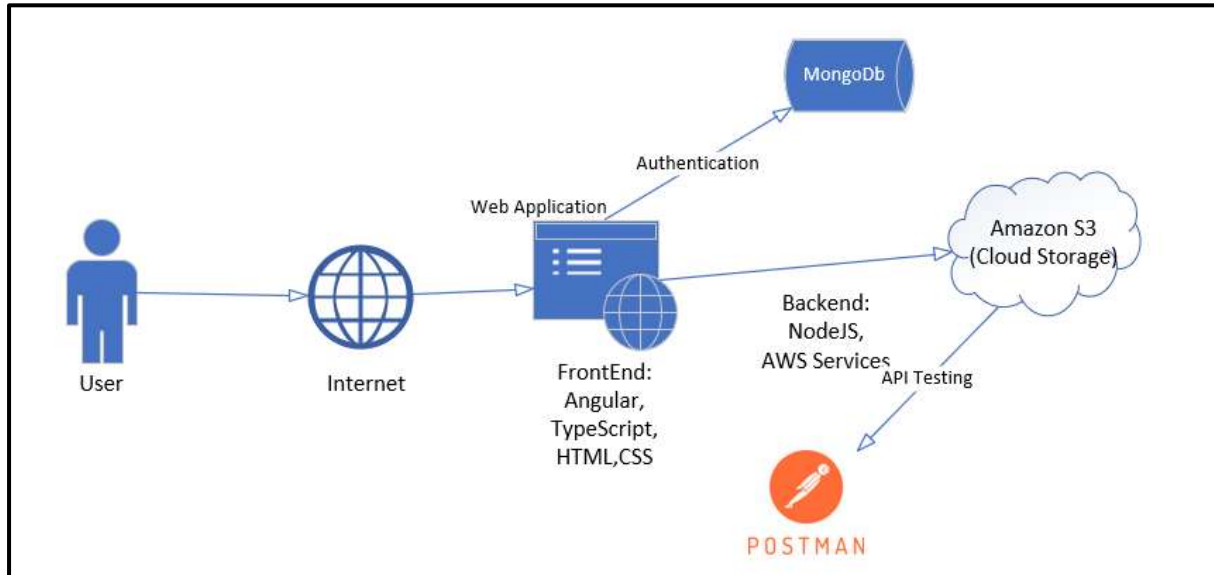


Figure 23 Implementation Diagram

- In this system, used Mongo DB Database for User storage and access permission, and also for authentication purpose.
- The main aim of the system is to upload files, which are stored on Blob storage.
- Here, project uses Amazon S3 (Simple Storage Service) for storing and retrieving all files and folders.
- In this project, created middle layer of APIs (Application Programming Interfaces) to connect blob storage and client.
- Also, created Upload functionality of files and folders which will make changes from User Interface to Amazon S3, with the help of Amazon Web Services SDK (Software Development Kit) and Nodejs.
- For Code management, used GitHub repository functionalities.
- Also, implemented CI/CD (Continuous Integration and Continuous Development) pipeline to enhance the work flow, using Azure DevOps.
- Here, API is the key implementation of backend, which I will be explaining later in the report.

5.2.1 Key Topics

1. Multer S3:

- Streaming multer storage engine for AWS S3.
- This is the node module required to use AWS S3 programmatically.
- Project uses this to upload files in S3.



Figure 24 Node Module-Multer S3

2. Main Structure:

- It is worth to show the folder structure of backend, which consists of mainly
 - Config: To setup the AWS 3 credentials
 - Controllers: Middleware of API to check constraints
 - Models: Database of the system
 - Routers: It displays the whole system and API's routes

Name	Date modified	Type
 config	12-04-2021 10:19 PM	File folder
 controllers	13-04-2021 03:41 PM	File folder
 models	12-04-2021 11:21 PM	File folder
 routers	13-04-2021 03:41 PM	File folder

Figure 25 Backend-Project Structure

3. Rest API for Amazon S3:

- System's main heart is the APIs, that upload, create, read and deletes the data and files.
- Created and Implemented API layer between Client and Amazon S3 for data sending and receiving.
- Also, these APIs can be used by company in another project also.

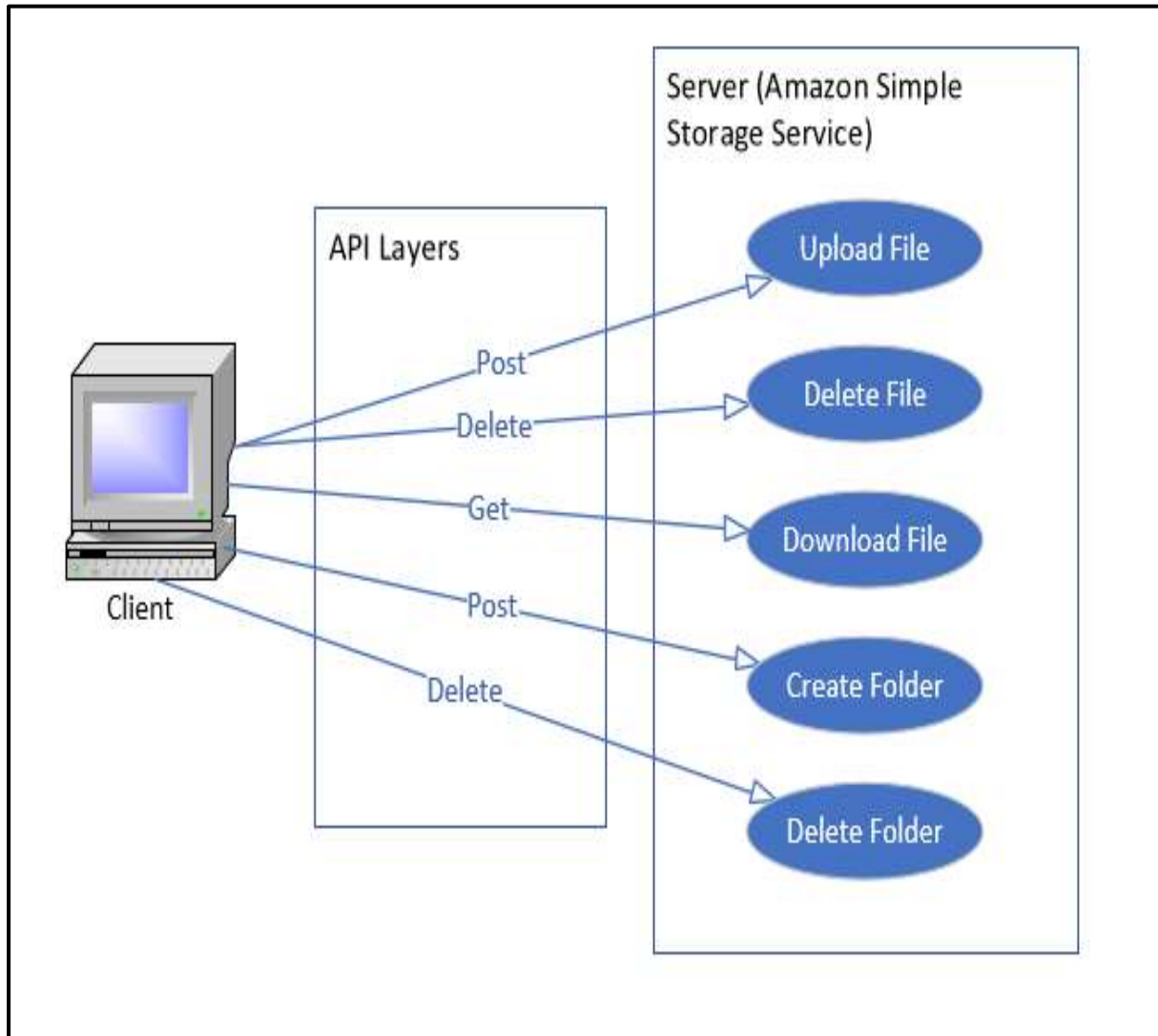


Figure 26 Application Programming Layer

4. Database

- As mentioned above, this project uses MongoDB for the database part.
- System has only one database, which is of user model as of now, as the files and files related metadata are stored in Amazon S3.
- User model consists of these many fields, which is mentioned in the below image.

Key	Value	Type
> (1) ObjectId("606c3e6385b45d7190eff503")	{ 9 fields }	Object
> (2) ObjectId("606c42a9d11bb3237881d68d")	{ 9 fields }	Object
> (3) ObjectId("606c8f17d1dfa473b4fe2fa8")	{ 9 fields }	Object
> (4) ObjectId("606d3064f20ad12c78faa79e")	{ 9 fields }	Object
> (5) ObjectId("606eb7f5d29oce4880412550")	{ 9 fields }	Object
> (6) ObjectId("607018f86bce4870548c478d")	{ 9 fields }	Object
> (7) ObjectId("60752c210535322db002b644")	{ 9 fields }	Object
> (8) ObjectId("60752cab8f7c25776ccde295")	{ 11 fields }	Object
> (9) ObjectId("607558f961212b5cae05c11dd")	{ 11 fields }	Object
> (10) ObjectId("60756f310625363378a07ad9")	{ 11 fields }	Object
> (11) ObjectId("60757bfd57caad509cae5527")	{ 11 fields }	Object
▼ (12) ObjectId("60757cc857caad509cae5528")	{ 11 fields }	Object
_id	ObjectId("60757cc857caad509cae5528")	ObjectId
isAdmin	0	Int32
readAccess	1	Int32
writeAccess	1	Int32
fullName	Krishnakumar Sir	String
email	kksir@tcs.com	String
password	\$2a\$10\$uUq2Plhgu5YeeRndus60puidAV7Qobczs/Wzs3d.y1.nbHcqE...	String
createdAt	2021-04-13 11:13:12.643Z	Date
updatedAt	2021-04-13 11:13:12.643Z	Date
saltSecret	\$2a\$10\$uUq2Plhgu5YeeRndus60pu	String
__v	0	Int32

Figure 27 Robo3T Database Snapshot

5. Amazon S3 Metadata:

- The system uploads the 7 types of metadata to S3, which is visible in the below image.
- While user is uploading the file, we fetch the user's name and the user id from the database, also fetch the remarks and description data from the user interface, together all these things will be stored in S3 object
- Also, there is one unique identifier field is there, named docid, which is auto generated using uuid, Nodejs module.

Metadata (7) Edit		
Metadata is optional information provided as a name-value (key-value) pair. Learn more		
Type	Key	Value
System defined	Content-Type	application/octet-stream
User defined	x-amz-meta-remarks	UK2
User defined	x-amz-meta-docid	0a5549cc-2209-43b8-8f4c-5fa86cabe09e
User defined	x-amz-meta-isfolder	0
User defined	x-amz-meta-owner	tirth shah
User defined	x-amz-meta-userid	606eb7f5d29ace4880412550
User defined	x-amz-meta-documenttype	Insurance

Figure 28 Metadata of Amazon S3

6. Postman:

- Postman is a collaboration platform for API development. Postman's features simplify each step of building an API and streamline collaboration so you can create better APIs—faster.
- In the initial stage, when we don't have user interface, used Postman for testing the APIs, which is really makes the work easier in that situation.

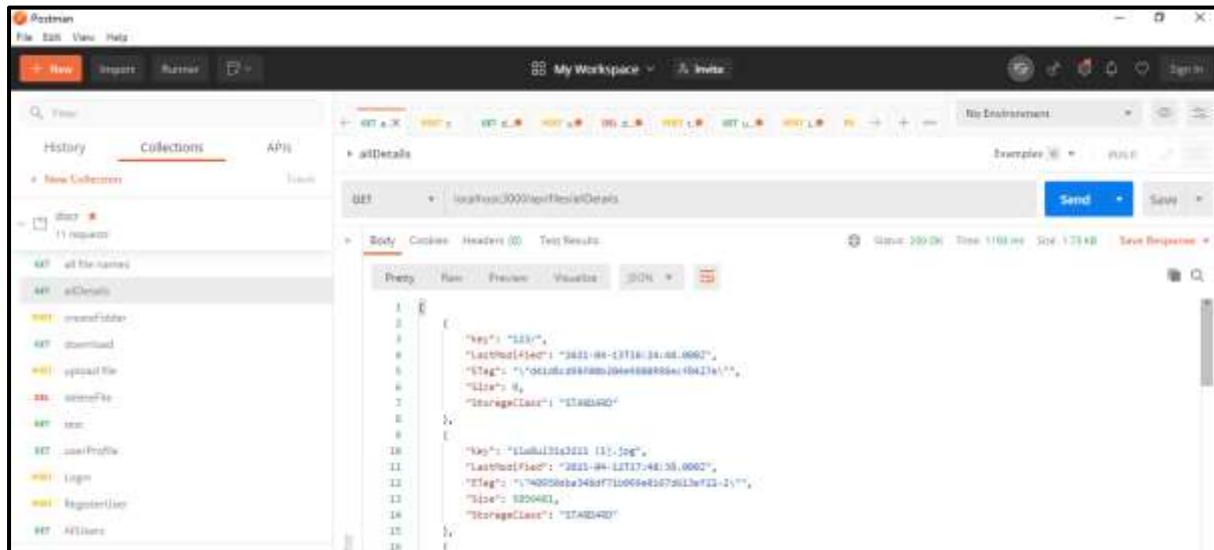


Figure 29 Postman-API Testing

7. Delete Functionality:

- System whenever calls the delete API, it will move the particular file to another S3 bucket.
- From that S3 bucket, it will be automatically deleting after 7 days, using programmatic script.
- Here is the snapshot of that script:

```
aws s3 ls BUCKETNAME/ | while read -r line;
do
createDate=`echo $line|awk '{print $1" "$2}'`
createDate=`date -d"$createDate" +%s`
olderThan=`date --date "7 days ago" +%s`
if [[ $createDate -lt $olderThan ]]
then
fileName=`echo $line|awk '{print $4}'`

if [[ $fileName != "" ]]
then
aws s3 rm BUCKETNAME/$fileName
fi
fi
done;
```

Figure 30 Deleted File Action

5.3 Code Management- Azure DevOps

- Azure DevOps is set of services which is used to:
 1. Plan Smarter
 2. Collaborate better
 3. Ship Faster
- Azure DevOps is managed and developed by Microsoft.
- It provides a set of integrated features that you can access through your web browser.

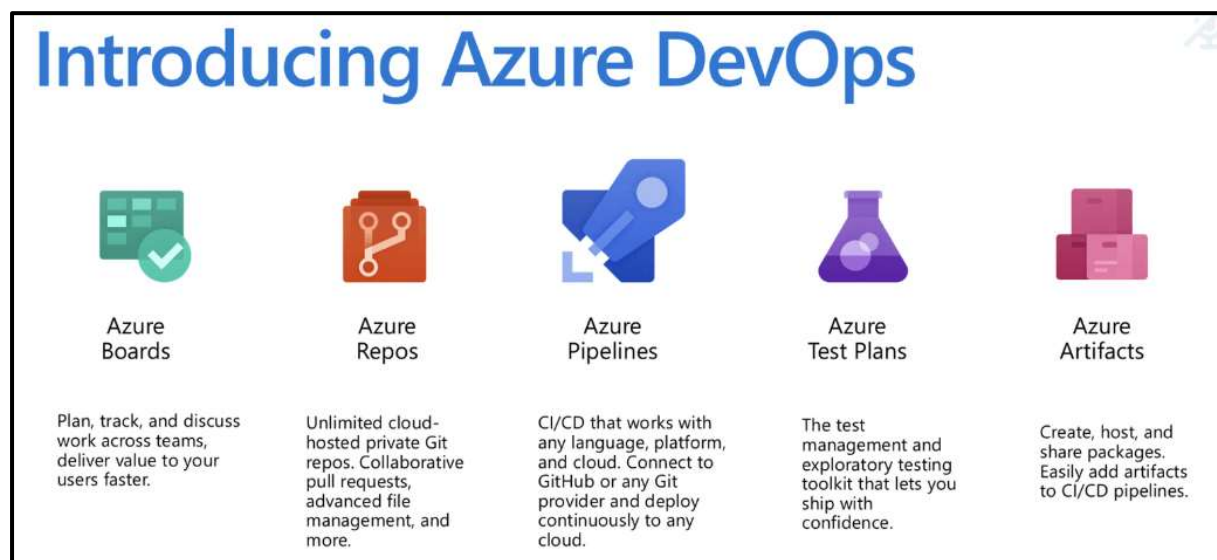


Figure 31 Azure DevOps

Azure DevOps Services:

1. Boards

Azure Boards is the service for tracking the backlog. Create and track work items, plan sprints, define dashboards, measure, and so on.

2. Repos-Used this for Code Management

Azure Repos is the service for the source code repository. It supports Git and TFVC (Team Foundation Version Control). It supports any Git client and integrations via REST API and webhooks.

3. Pipeline

Azure Pipelines is the service to automate builds and deployments with tasks and reproducible steps for consistency of CI/CD pipeline stages.

4. Test Plans

Azure Test Plans is the service to create and execute test plans, improving the quality of the code and validating results throughout the pipeline, with end-to-end traceability.

5. Artifacts

Azure Artifacts is the service for package repository.

5.3.1 Setup of Azure DevOps

1. Project Window:

- This is the main landing page of the Azure DevOps.
- From here, we can make the projects for the various teams.
- Our project tab is in the below image, named DocR.

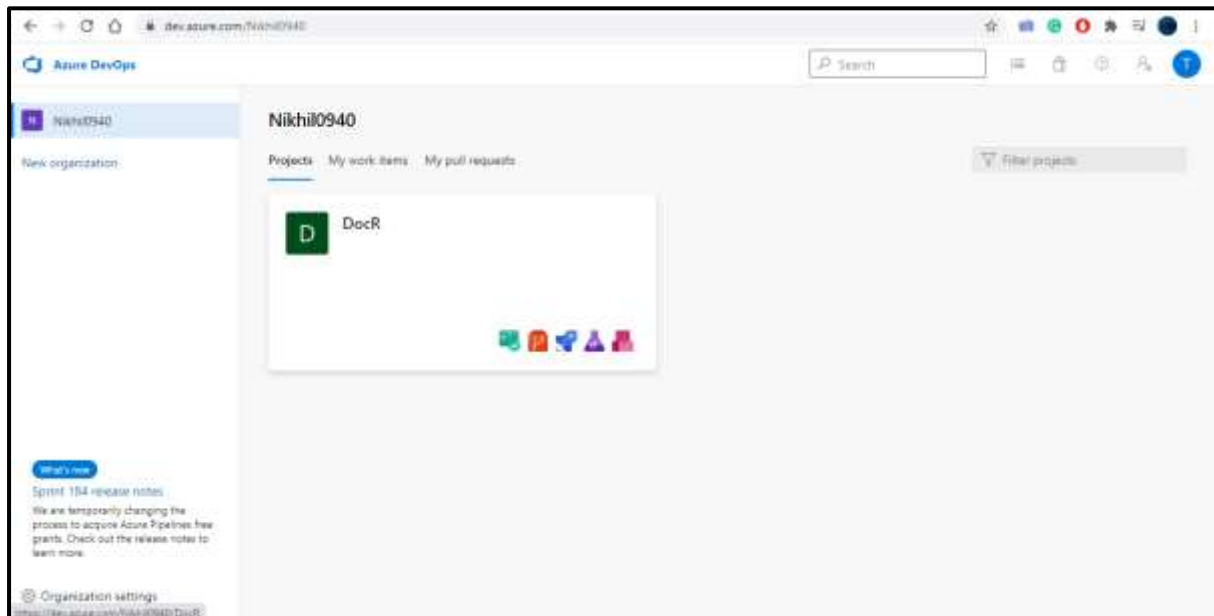


Figure 32 Azure Setup

2. Project Home Page:

- It is the summary page of the project.
- In the left panel, we can see the Azure DevOps services, which one can use for the betterment of the project.
- It also shows the project stats in the right side.

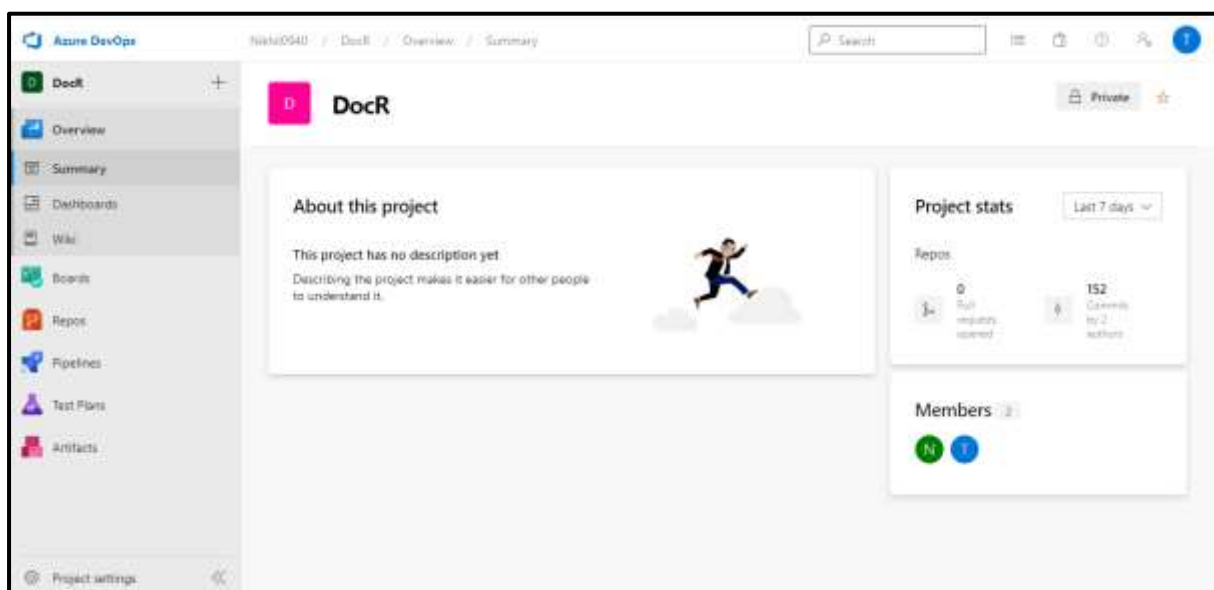


Figure 33 Project Home Window

3. Repos Tab:

- This window consists of all the source code of the project.
- Here, it displays all the commits, pushes and pull requests.
- In our project, we migrate our GitHub repo to Azure DevOps repos.

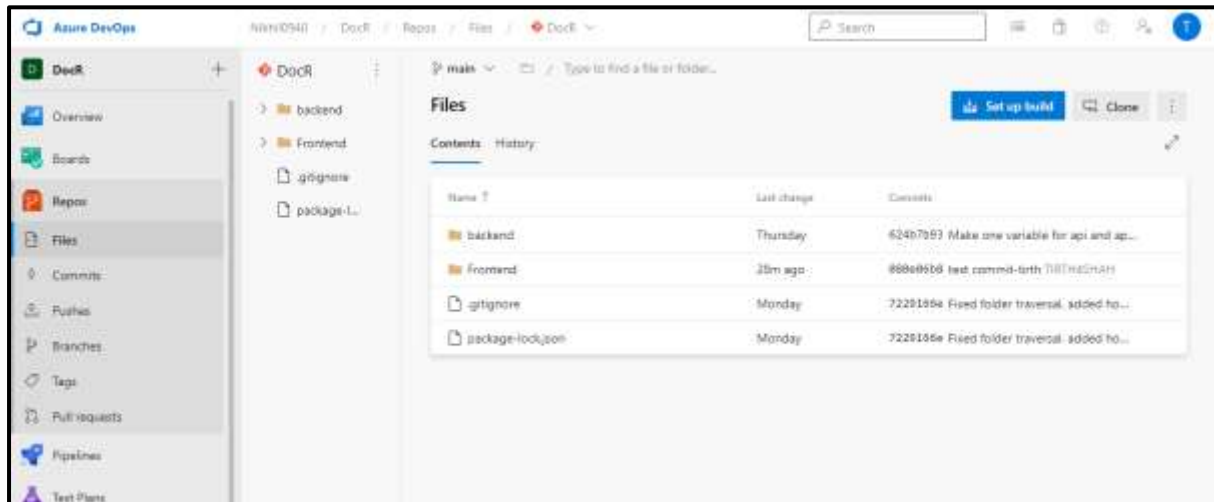


Figure 34 Azure Repository

4. Commits Window:

- It shows the status of the commit from respective branches of the repos.
- Also, we can search the commit id from the search bar to see the detailed changes in case of error handling.

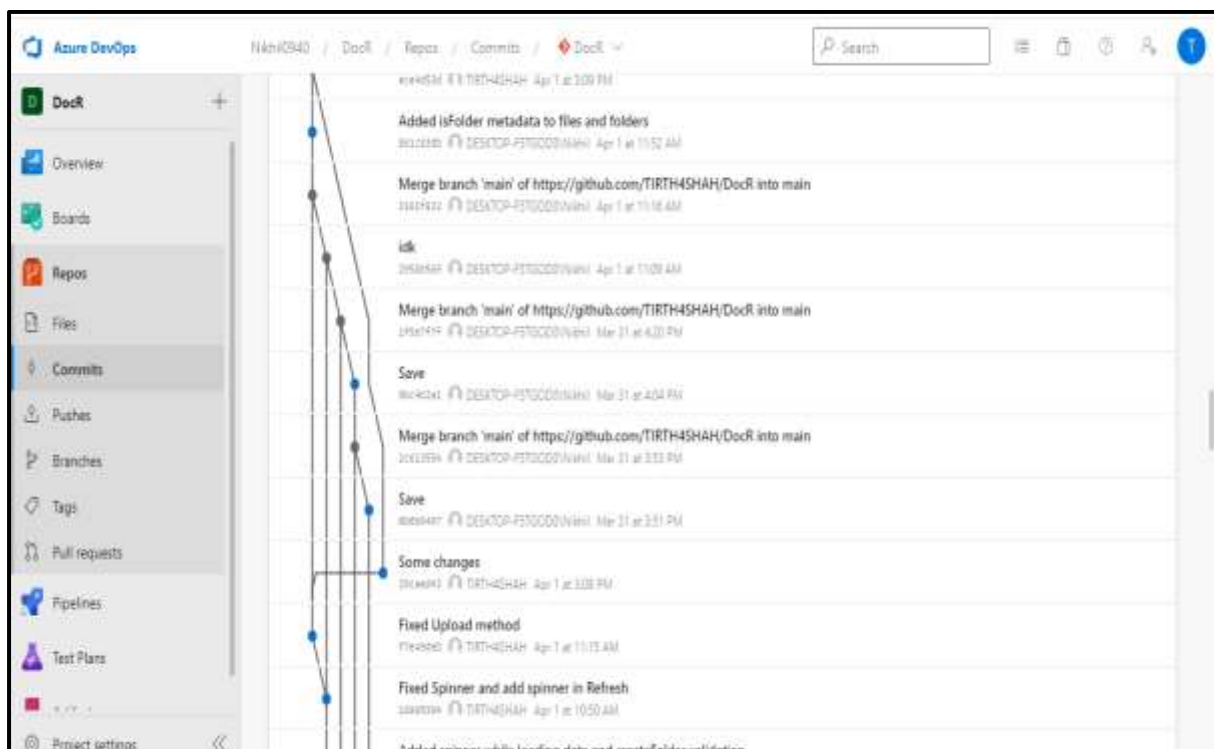


Figure 35 Commit History

5.4 Docker Containerization

- Docker is an open-source DevOps tool designed to help developers and operations guys to streamline application development and deployment.
- By dockerizing an application means deploying and running an application using containers.
- Containers allow a developer to package up an application with all of the parts it needs, such as libraries and other dependencies, and deploy it as one package.
- By doing so application can be deployed on any target environment/machine regardless of any customized settings that machine might have that could differ from the machine used for writing and testing the code.

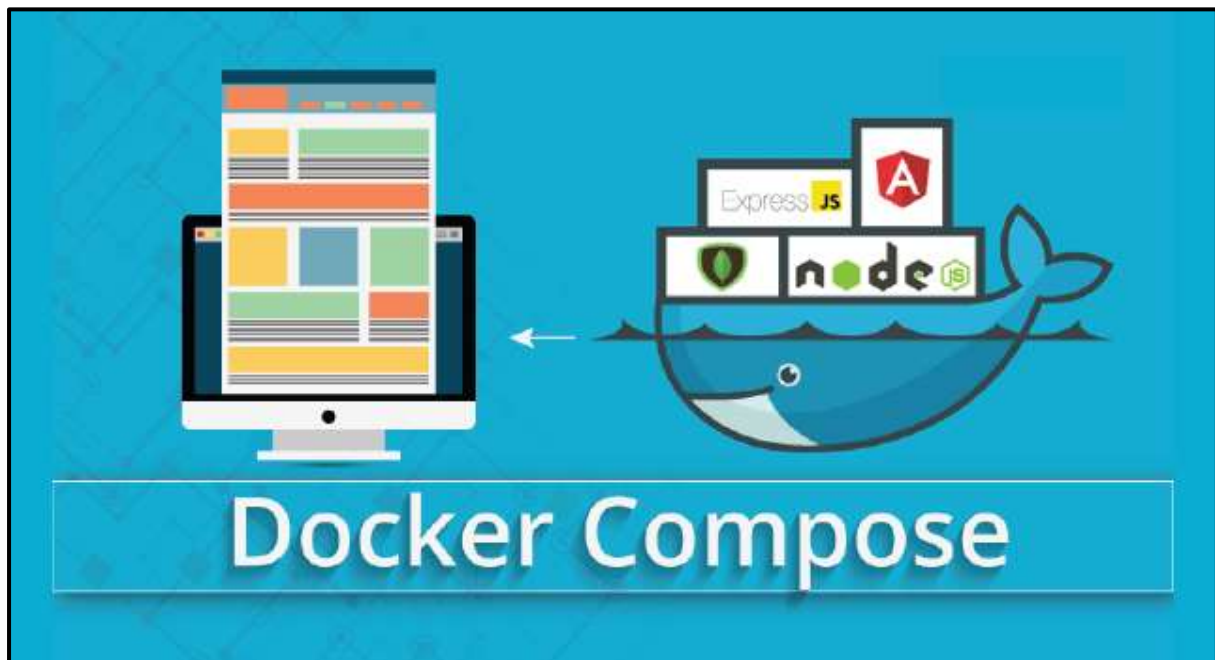


Figure 36 Docker Introduction

1. Docker Toolbox

- Docker Toolbox is an installer for quick setup and launch of a Docker environment on Mac and Windows systems.
- From here, we can interact with the docker client.
- Also, we can execute docker related command here.

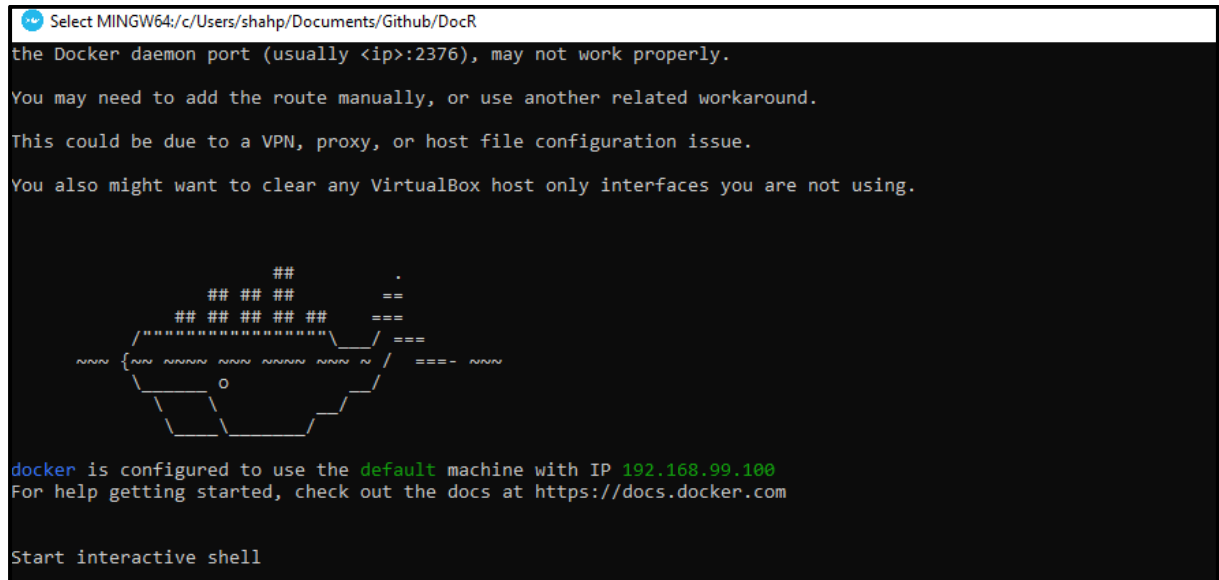


Figure 37 Docker Toolbox

2. Docker Containers

- For this project, I made 3 containers, which are Frontend, Backend and Database.
- First of all, we have to create DockerFile for each, in that we fetch the image of particular framework from the docker hub.
- After that, declare the commands that are necessary to run the system.
- Using docker ps command, we can see the running containers of the project.

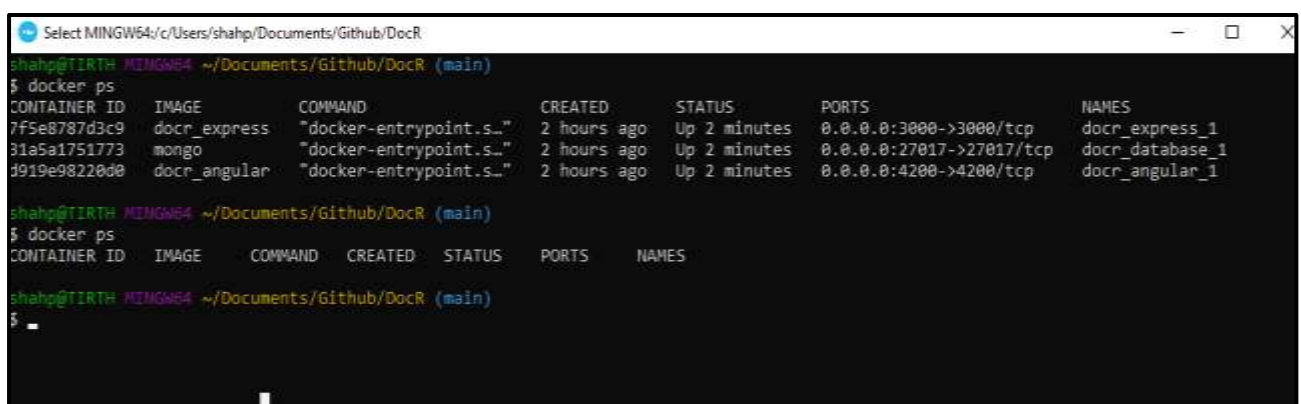
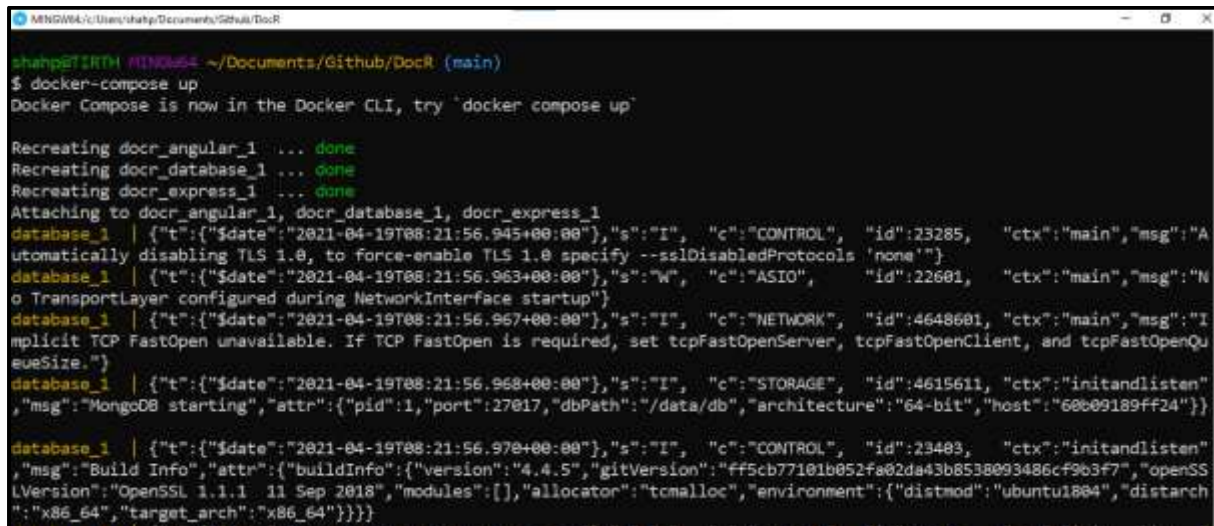


Figure 38 Containers

3. Docker Compose

- Compose is a tool for defining and running multi-container Docker applications. With Compose, you use a YAML file to configure your application's services. Then, with a single command, you create and start all the services from your configuration.
- Here, in this project, we combined two Docker files which are frontend and backend into one docker compose file.
- From docker compose file, one can run the multiple containers with a single command.



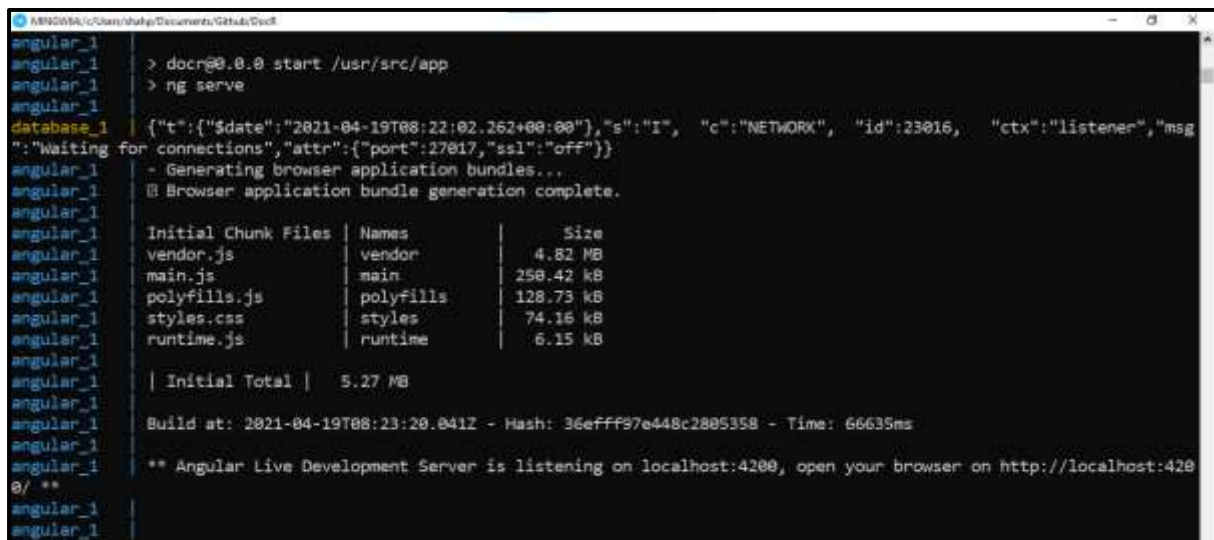
```

shahp@TIRTH: ~/Documents/Github/DocR (main)
$ docker-compose up
Docker Compose is now in the Docker CLI, try 'docker compose up'

Recreating docr_angular_1 ... done
Recreating docr_database_1 ... done
Recreating docr_express_1 ... done
Attaching to docr_angular_1, docr_database_1, docr_express_1
database_1 | {"t":{"$date":"2021-04-19T08:21:56.945+00:00"},"s":"I", "c":"CONTROL", "id":23285, "ctx":"main","msg":"A
database_1 | {"t":{"$date":"2021-04-19T08:21:56.963+00:00"},"s":"W", "c":"ASIO", "id":22601, "ctx":"main","msg":"N
database_1 | {"t":{"$date":"2021-04-19T08:21:56.967+00:00"},"s":"I", "c":"NETWORK", "id":4648601, "ctx":"main","msg":"I
database_1 | {"t":{"$date":"2021-04-19T08:21:56.968+00:00"},"s":"I", "c":"STORAGE", "id":4615611, "ctx":"initandlisten"
database_1 | {"t":{"$date":"2021-04-19T08:21:56.970+00:00"},"s":"I", "c":"CONTROL", "id":23403, "ctx":"initandlisten"

```

Figure 39 Docker Compose Up



```

angular_1 > docr@0.0.0 start /usr/src/app
angular_1 > ng serve
database_1 {"t":{"$date":"2021-04-19T08:22:02.262+00:00"},"s":"I", "c":"NETWORK", "id":23016, "ctx":"listener","msg":
angular_1 - Generating browser application bundles...
angular_1 Browser application bundle generation complete.

Initial Chunk Files | Names | Size
vendor.js | vendor | 4.82 MB
main.js | main | 250.42 kB
polyfills.js | polyfills | 128.73 kB
styles.css | styles | 74.16 kB
runtime.js | runtime | 6.15 kB

| Initial Total | 5.27 MB

Build at: 2021-04-19T08:23:20.041Z - Hash: 36efff97e448c2805358 - Time: 66635ms
** Angular Live Development Server is listening on localhost:4200, open your browser on http://localhost:4200/ **

```

Figure 40 Multi-Containers

Chapter 6 UML Diagrams

6.1 System Design

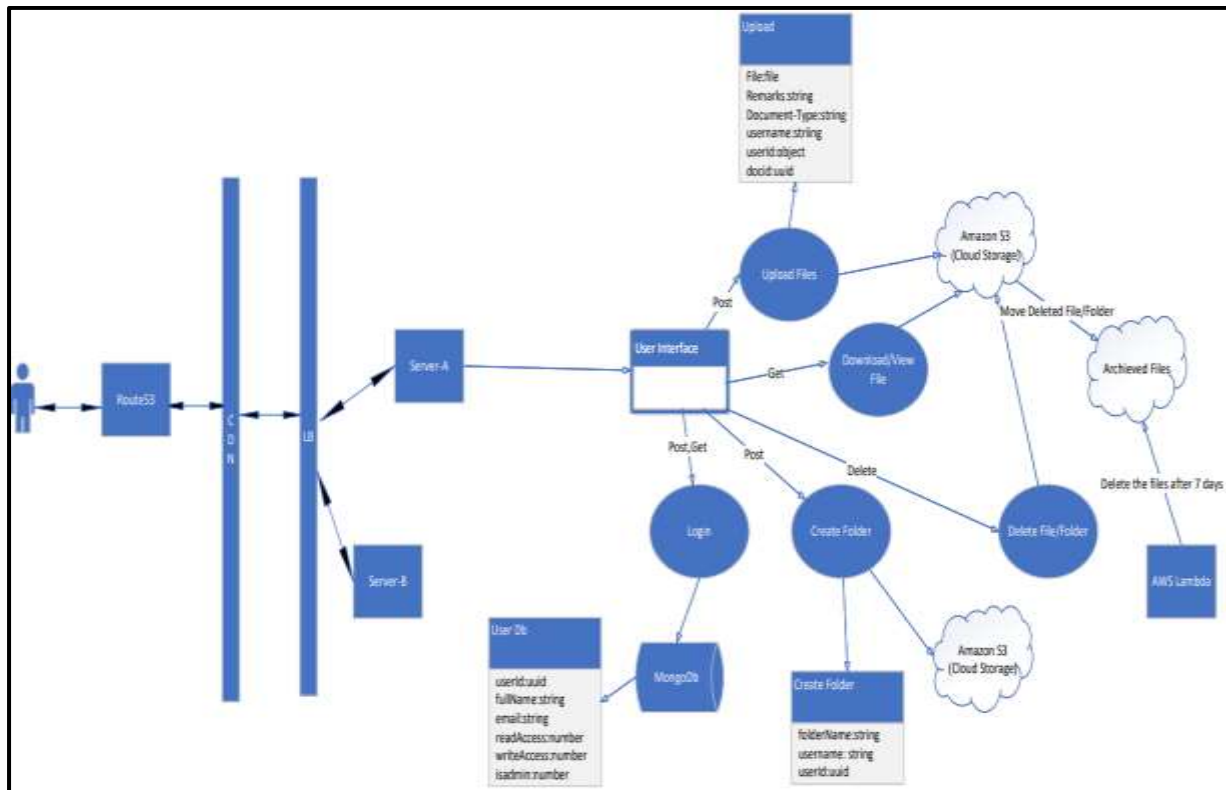


Figure 41 System Design

6.2 Use Case Diagram

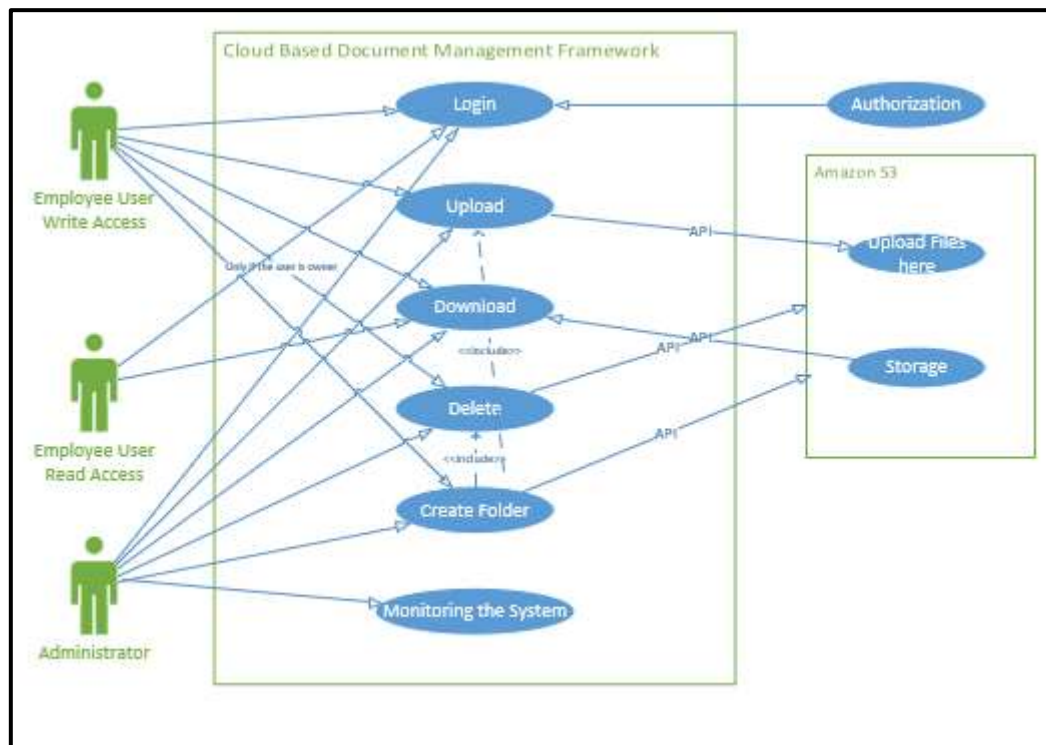


Figure 42 Use Case

6.3 Class Diagram

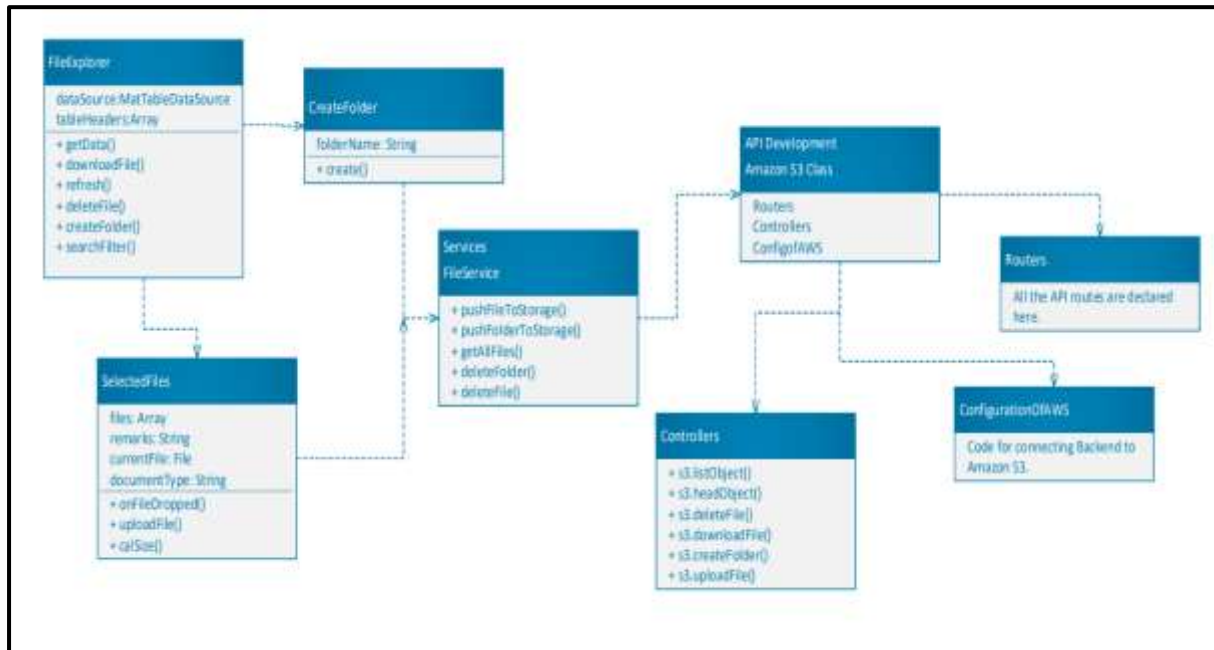


Figure 43 Class Diagram

6.4 Activity Diagram

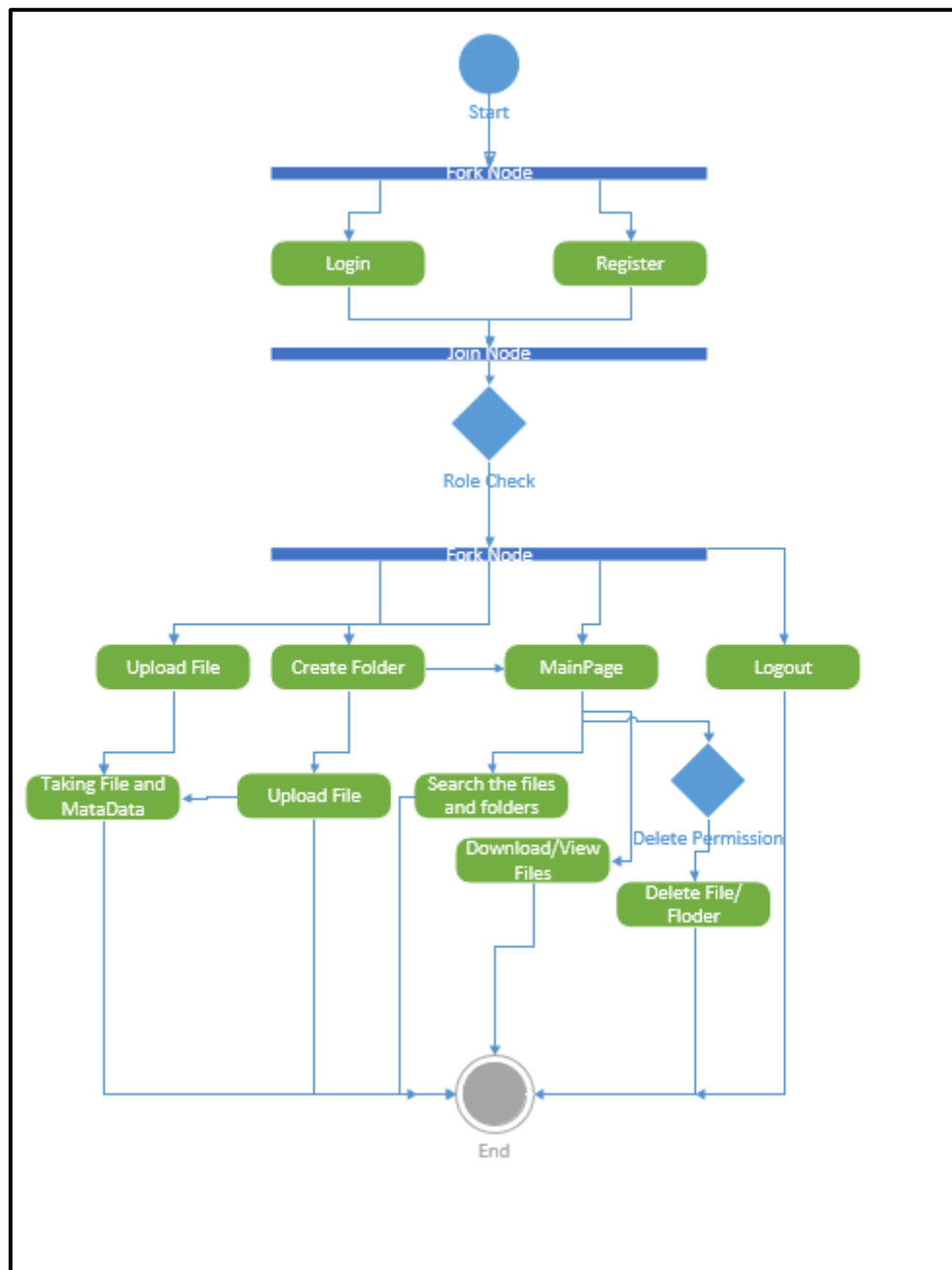


Figure 44 Activity Diagram

Chapter 7 Conclusion

- This project will simplify the work of the employees of the respective business unit.
- It gave us exposure to Full Stack Development and also give us understanding that how to work in a team for the project, which will be extremely useful in industry.
- A robust file repository system that supports simultaneous uploads of multiple files, structuring of files into infinite layers of folders/subfolders, and access control using a user login system that also manages privileges and permitted actions of users.
- A Docker image of the application that allows it to be run on any operating system in the way it was intended.

Chapter 8 References

- <https://docs.aws.amazon.com/AWSJavaScriptSDK/latest/AWS/S3.html>
- <https://angular.io/guide/architecture>
- <https://expressjs.com/en/5x/api.html>
- <https://docs.mongodb.com/manual/crud/>
- <https://www.tcs.com/>
- <https://docs.microsoft.com/en-us/azure/devops/?view=azure-devops>